



POLITEKNIK MANUFAKTUR BANDUNG

Jalan Kanayakan 21 Tlp/Fax. +62 (022) 7312564 Bandung 40275
Telp. (022)2500241 Faks. (022)2502649 <https://polman-bandung.ac.id/>



LAPORAN TUGAS MANDIRI 2 PEMROGRAMAN BERBASIS OBJEK (Minggu ke-1)



Nama : Syaroful Haidar Sudaryono
NIM : 225443051
Kelas : 2AEC2



DAFTAR ISI

DAFTAR ISI.....	1
PRAKTIKUM HARI KE – 2.....	2
(Sistem Manajemen Transportasi Online).....	2
1. Tujuan dan Penjelasan Kelas.....	2
2. Hasil Eksekusi (Demonstrasi Program).....	3
3. Analisis OOP	4
1.3. Kelas Dasar & Enkapsulasi	4
2.3. Pewarisan (Inheritance) & Polimorfisme	4
3.3. Interface & Sealed Class	4
LAMPIRAN	5
1. LAMPIRAN SOURCE CODE.....	5



PRAKTIKUM HARI KE – 2

(Sistem Manajemen Transportasi Online)

1. Tujuan dan Penjelasan Kelas

- **Tujuan Soal:** Mengimplementasikan konsep dasar OOP, Enkapsulasi, Pewarisan, Polimorfisme, dan integrasi sistem secara bertahap pada studi kasus Transportasi Online.
- **Penjelasan Kelas:** Kelas Vehicle beserta turunannya (Car, Motorcycle, Truck) mengelola data dan kalkulasi tarif. Driver dan Customer menyimpan entitas pengguna. Order menghubungkan pengguna, rute, dan status pesanan (menggunakan *sealed class* OrderStatus). Payment dan *interface* PaymentMethod menangani transaksi. Seluruh entitas ini dikelola secara terpusat oleh kelas TransportSystem.



2. Hasil Eksekusi (Demonstrasi Program)

```
=== Go-Transport 2024 ===

=== TARIF KENDARAAN UNTUK JARAK 20 KM ===
Mobil: Rp58.000
Motor: Rp33.000
Truk: Rp80.000

=== DAFTAR KENDARAAN ===
Nomor polisi : B 1234 XYZ
Kendaraan   : Toyota Innova (2021)
Jenis       : Mobil
Tersedia    : Ya
Bahan bakar : Bensin
Jumlah pintu : 4
|
Nomor polisi : D 5678 ABC
Kendaraan   : Honda Beat (2022)
Jenis       : Motor
Tersedia    : Ya
Kapasitas   : 125 cc
Helm        : Tersedia
|
Nomor polisi : E 9012 DEF
Kendaraan   : Hino Dutro (2020)
Jenis       : Truk
Tersedia    : Ya
Kapasitas   : 5.0 ton
Jumlah sumbu : 2

=== DAFTAR PENGEMUDI ===
ID driver   : D001
Nama driver : Andi
Telepon     : 08123456789
Status aktif : Ya
Nomor polisi : B 1234 XYZ
Kendaraan   : Toyota Innova (2021)
Jenis       : Mobil
Tersedia    : Ya
Bahan bakar : Bensin
Jumlah pintu : 4
|
ID driver   : D002
Nama driver : Budi
Telepon     : 08129876543
Status aktif : Ya
Nomor polisi : D 5678 ABC
Kendaraan   : Honda Beat (2022)
Jenis       : Motor
Tersedia    : Ya
Kapasitas   : 125 cc
Helm        : Tersedia
|
ID driver   : D003
Nama driver : Citra
Telepon     : 08125678901
Status aktif : Ya
Nomor polisi : E 9012 DEF
```

```
Kendaraan   : Hino Dutro (2020)
Jenis       : Truk
Tersedia    : Ya
Kapasitas   : 5.0 ton
Jumlah sumbu : 2

=== DAFTAR PELANGGAN ===
ID pelanggan : C001
Nama         : Dewi
Telepon      : 08134567890
Email        : dewi@email.com
Saldo        : Rp100.000
|
ID pelanggan : C002
Nama         : Eko
Telepon      : 08135678901
Email        : eko@email.com
Saldo        : Rp50.000
|
ID pelanggan : C003
Nama         : Fani
Telepon      : 08136789012
Email        : fani@email.com
Saldo        : Rp200.000

=== DAFTAR PESANAN ===
ID pesanan   : ORD-001
Pelanggan    : Dewi
Pengemudi    : Andi
Rute         : Kampus A -> Mall B
Jarak        : 12.0 km
Total tarif  : Rp38.000
Status       : Menunggu
|
ID pesanan   : ORD-002
Pelanggan    : Eko
Pengemudi    : Budi
Rute         : Stasiun -> Kantor
Jarak        : 8.0 km
Total tarif  : Rp15.000
Status       : Menunggu
|
ID pesanan   : ORD-003
Pelanggan    : Fani
Pengemudi    : Citra
Rute         : Gudang -> Pelabuhan
Jarak        : 25.0 km
Total tarif  : Rp97.500
Status       : Menunggu
```



3. Analisis OOP

1.3. Kelas Dasar & Enkapsulasi

- **val vs var:** `val` bersifat *immutable* (tidak dapat diubah setelah inisialisasi, contoh: `plateNumber`), sedangkan `var` bersifat *mutable* (dapat diperbarui, contoh: `balance`).
- **private set vs private val:** `private set` memungkinkan data dibaca secara bebas dari luar kelas melalui *getter* publik, namun modifikasinya dibatasi hanya dari dalam kelas itu sendiri. Hal ini krusial untuk melindungi integritas status pembayaran (`_isPaid`) dan status pesanan.
- **Validasi:** Validasi metode pembayaran (misal pengecekan 16 digit kartu kredit) berfungsi sebagai gerbang keamanan untuk mencegah sistem memproses input data yang tidak valid.

2.3. Pewarisan (Inheritance) & Polimorfisme

- **open class & super:** Kelas `Vehicle` diubah menjadi open class karena secara bawaan kelas Kotlin bersifat final. Pemanggilan `super.displayInfo()` pada sub-kelas digunakan agar tidak perlu menulis ulang logika pencetakan data dasar dari kelas induk.
- **Polimorfisme pada calculateFare():** Metode ini di-*override* karena mobil, motor, dan truk memiliki parameter tarif dasar dan per kilometer yang berbeda. Konsep ini juga terlihat pada `displayAllVehicles()`, di mana program otomatis mengeksekusi metode spesifik sesuai wujud asli kendaraannya meski disimpan secara umum bertipe `Vehicle`.

3.3. Interface & Sealed Class

- **Interface:** `PaymentMethod` menggunakan *interface* karena sistem hanya membutuhkan standarisasi fungsi pemrosesan tanpa memerlukan penyimpanan data turunan (*state*).
- **Sealed Class:** `PaymentResult` dan `OrderStatus` menggunakan *sealed class* agar hasilnya bersifat pasti. Kompiler menyadari seluruh skenario (ekshaustif), serta mendukung penyimpanan parameter dinamis (seperti alasan pada status `Cancelled`).
- **Smart Casting (as?):** Operator `as?` digunakan untuk konversi tipe data secara aman. Apabila objek tidak sesuai, sistem mengembalikan `null` alih-alih menghentikan program secara paksa.



LAMPIRAN

1. LAMPIRAN SOURCE CODE

```
// =====  
// 1. Vehicle.kt  
// =====  
open class Vehicle(  
    val plateNumber: String,  
    val brand: String,  
    val model: String,  
    val year: Int,  
    var isAvailable: Boolean = true  
) {  
    open fun getType(): String = "Kendaraan Umum"  
  
    open fun calculateFare(distanceKm: Double): Double {  
        return 5000 + (distanceKm * 2000)  
    }  
  
    open fun displayInfo() {  
        println("Nomor polisi : $plateNumber")  
        println("Kendaraan : $brand $model ($year)")  
        println("Jenis : ${getType()}")  
        println("Tersedia : ${if (isAvailable) "Ya" else "Tidak"}")  
    }  
}  
  
// =====  
// 2. Car.kt  
// =====  
class Car(  
    plateNumber: String,  
    brand: String,  
    model: String,  
    year: Int,  
    val fuelType: String,
```



```
        val numberOfDoors: Int
    ) : Vehicle(plateNumber, brand, model, year) {

        override fun getType(): String = "Mobil"

        override fun calculateFare(distanceKm: Double): Double {
            return 8000 + (distanceKm * 2500)
        }

        override fun displayInfo() {
            super.displayInfo()
            println("Bahan bakar : $fuelType")
            println("Jumlah pintu : $numberOfDoors")
        }
    }
}

// =====
// 3. Motorcycle.kt
// =====
class Motorcycle(
    plateNumber: String,
    brand: String,
    model: String,
    year: Int,
    val engineCapacity: Int,
    val hasHelmet: Boolean
) : Vehicle(plateNumber, brand, model, year) {

    override fun getType(): String = "Motor"

    override fun calculateFare(distanceKm: Double): Double {
        return 3000 + (distanceKm * 1500)
    }
}
```

```
        override fun displayInfo() {
```



```
        super.displayInfo()
        println("Kapasitas      : $engineCapacity cc")
        println("Helm          : ${if (hasHelmet) "Tersedia" else "Tidak ada"}")
    }
}

// =====
// 4. Truck.kt
// =====
class Truck(
    plateNumber: String,
    brand: String,
    model: String,
    year: Int,
    val loadCapacity: Double,
    val numberOfAxles: Int
) : Vehicle(plateNumber, brand, model, year) {

    override fun getType(): String = "Truk"

    override fun calculateFare(distanceKm: Double): Double {
        return 10000 + (distanceKm * 3500)
    }

    override fun displayInfo() {
        super.displayInfo()
        println("Kapasitas      : $loadCapacity ton")
        println("Jumlah sumbu : $numberOfAxles")
    }
}

// =====
// 5. Driver.kt
// =====
class Driver(
```



```
    val id: String,
    val name: String,
    val phone: String,
    val vehicle: Vehicle,
    var isActive: Boolean = true
) {
    fun displayInfo() {
        println("ID driver    : $id")
        println("Nama driver   : $name")
        println("Telepon      : $phone")
        println("Status aktif : ${if (isActive) "Ya" else "Tidak"}")
        vehicle.displayInfo()
    }

    fun acceptOrder(): Boolean {
        return isActive && vehicle.isAvailable
    }
}

// =====
// 6. Customer.kt
// =====
class Customer(
    val id: String,
    val name: String,
    val phone: String,
    val email: String,
    var balance: Double = 0.0
) {
    fun displayInfo() {
        println("ID pelanggan : $id")
        println("Nama        : $name")
        println("Telepon     : $phone")
        println("Email       : $email")
        println("Saldo       : ${formatRupiah(balance)}")
    }
}
```



```
}  
  

```

```
fun topUp(amount: Double) {  
    balance += amount  
}
```

```
fun canPay(amount: Double): Boolean {  
    return balance >= amount  
}  
}
```

```
fun formatRupiah(amount: Double): String {  
    return "Rp%,d".format(amount.toLong()).replace(',', '.')  
}
```

```
// =====  
// 7. OrderStatus.kt  
// =====
```

```
sealed class OrderStatus {  
    abstract fun display(): String  
    open fun isFinal(): Boolean = this is Completed || this is Cancelled
```

```
    object Waiting : OrderStatus() {  
        override fun display() = "Menunggu"  
    }
```

```
    object OnGoing : OrderStatus() {  
        override fun display() = "Berjalan"  
    }
```

```
    object Completed : OrderStatus() {  
        override fun display() = "Selesai"  
    }
```

```
    data class Cancelled(val reason: String) : OrderStatus() {
```



```
        override fun display() = "Dibatalkan: $reason"
    }
}
```

```
// =====
// 8. Order.kt
// =====
```

```
class Order(
    val id: String,
    val customer: Customer,
    val driver: Driver,
    val pickupLocation: String,
    val destination: String,
    val distanceKm: Double
) {
    var status: OrderStatus = OrderStatus.Waiting
    private set
```

```
    val totalFare: Double = driver.vehicle.calculateFare(distanceKm)
```

```
    fun startTrip(): Boolean {
        if (status is OrderStatus.Waiting) {
            status = OrderStatus.OnGoing
            return true
        }
        return false
    }
```

```
    fun completeTrip(): Boolean {
        if (status is OrderStatus.OnGoing) {
            status = OrderStatus.Completed
            driver.vehicle.isAvailable = true
            return true
        }
        return false
    }
```



```
}  
  

```

```
fun cancelTrip(reason: String): Boolean {  
    if (!status.isFinal()) {  
        status = OrderStatus.Cancelled(reason)  
        driver.vehicle.isAvailable = true  
        return true  
    }  
    return false  
}
```

```
fun displayOrder() {  
    println("ID pesanan : $id")  
    println("Pelanggan : ${customer.name}")  
    println("Pengemudi : ${driver.name}")  
    println("Rute : $pickupLocation -> $destination")  
    println("Jarak : $distanceKm km")  
    println("Total tarif : ${formatRupiah(totalFare)}")  
    println("Status : ${status.display()}")  
}
```

```
}  
  

```

```
// =====  
// 9. PaymentResult.kt  
// =====
```

```
sealed class PaymentResult {  
    abstract fun display(): String  
  
    data class Success(val transactionId: String, val timestamp: String) :  
        PaymentResult() {  
        override fun display() = "Berhasil. ID transaksi: $transactionId"  
    }  
  
    data class Failed(val reason: String, val errorCode: Int) : PaymentResult() {  
        override fun display() = "Gagal. $reason (kode $errorCode)"  
    }  
}
```



```
}  
  
object Pending : PaymentResult() {  
    override fun display() = "Tertunda"  
}  
  
}
```

```
// =====  
// 10. PaymentMethod.kt  
// =====  
interface PaymentMethod {  
    val name: String  
    fun processPayment(amount: Double): PaymentResult  
    fun getFee(amount: Double): Double = 0.0  
}
```

```
// =====  
// 11. CreditCard.kt  
// =====  
class CreditCard(val cardNumber: String) : PaymentMethod {  
    override val name: String = "Kartu Kredit"  
  
    override fun getFee(amount: Double): Double = amount * 0.02  
  
    override fun processPayment(amount: Double): PaymentResult {  
        return if (cardNumber.length >= 16) {  
            PaymentResult.Success("CC-${System.currentTimeMillis()}",  
java.time.LocalDateTime.now().toString())  
        } else {  
            PaymentResult.Failed("Nomor kartu tidak valid", 400)  
        }  
    }  
}
```

```
// =====
```



```
// 12. QRIS.kt
// =====
class QRIS(val qrcode: String) : PaymentMethod {
    override val name: String = "QRIS"

    override fun getFee(amount: Double): Double = amount * 0.005

    override fun processPayment(amount: Double): PaymentResult {
        return if (qrcode.length >= 10) {
            PaymentResult.Success("QRIS-#{System.currentTimeMillis()}",
            java.time.LocalDateTime.now().toString())
        } else {
            PaymentResult.Failed("Kode QR tidak valid", 400)
        }
    }
}

// =====
// 13. Cash.kt
// =====
class Cash : PaymentMethod {
    override val name: String = "Tunai"

    override fun processPayment(amount: Double): PaymentResult {
        return PaymentResult.Success("CASH-#{System.currentTimeMillis()}",
        java.time.LocalDateTime.now().toString())
    }
}

// =====
// 14. Payment.kt
// =====
class Payment(
    val order: Order,
    val method: PaymentMethod
) {
```



```
    val amount: Double = order.totalFare
    var isPaid: Boolean = false
    private set

    fun processPayment(paidAmount: Double): PaymentResult {
        val totalDue = amount + method.getFee(amount)
        if (!isPaid && paidAmount >= totalDue) {
            val result = method.processPayment(amount)
            if (result is PaymentResult.Success) {
                isPaid = true
            }
            return result
        } else {
            return PaymentResult.Failed("Nomor pembayaran kurang dari tagihan  
${formatRupiah(totalDue)}", 400)
        }
    }

    fun displayPayment() {
        println("Tagihan      : ${formatRupiah(amount)}")
        println("Metode       : ${method.name}")
        println("Status Lunas : ${if (isPaid) "Sudah" else "Belum"}")
    }
}

// =====
// 15. TransportSystem.kt
// =====
class TransportSystem(val name: String) {
    private val vehicles = mutableListOf<Vehicle>()
    private val drivers = mutableListOf<Driver>()
    private val customers = mutableListOf<Customer>()
    private val orders = mutableListOf<Order>()
    private val payments = mutableListOf<Payment>()
```



```
val vehiclesList: List<Vehicle> get() = vehicles.toList()
```

```
fun addVehicle(v: Vehicle) { vehicles.add(v) }
```

```
fun addDriver(d: Driver) { drivers.add(d) }
```

```
fun addCustomer(c: Customer) { customers.add(c) }
```

```
fun findVehicle(plate: String): Vehicle? = vehicles.find { it.plateNumber.equals(plate, true) }
```

```
fun findDriver(id: String): Driver? = drivers.find { it.id.equals(id, true) }
```

```
fun findCustomer(id: String): Customer? = customers.find { it.id.equals(id, true) }
```

```
fun createOrder(customerId: String, driverId: String, pickup: String, dest: String, distance: Double): Order? {
```

```
    val cust = findCustomer(customerId) ?: return null
```

```
    val drv = findDriver(driverId) ?: return null
```

```
    if (!drv.acceptOrder()) {
```

```
        println("Driver tidak aktif atau kendaraan tidak tersedia.")
```

```
        return null
```

```
    }
```

```
    drv.vehicle.isAvailable = false
```

```
    val order = Order("ORD-00${orders.size + 1}", cust, drv, pickup, dest, distance)
```

```
    orders.add(order)
```

```
    return order
```

```
}
```

```
fun processPayment(orderId: String, method: PaymentMethod, paidAmount: Double): PaymentResult {
```

```
    val ord = orders.find { it.id == orderId } ?: return PaymentResult.Failed("Order tidak ditemukan", 404)
```

```
    val payment = Payment(ord, method)
```

```
    payments.add(payment)
```

```
    return payment.processPayment(paidAmount)
```

```
}
```

```
fun completeOrder(orderId: String): Boolean {
```

```
    val ord = orders.find { it.id == orderId } ?: return false
```



```
        return ord.completeTrip()
```

```
    }
```

```
    fun cancelOrder(orderId: String, reason: String): Boolean {
```

```
        val ord = orders.find { it.id == orderId } ?: return false
```

```
        return ord.cancelTrip(reason)
```

```
    }
```

```
    fun displayAllVehicles() {
```

```
        println("=== DAFTAR KENDARAAN ===")
```

```
        vehicles.forEach { it.displayInfo(); println() }
```

```
    }
```

```
    fun displayAllDrivers() {
```

```
        println("=== DAFTAR PENGEMUDI ===")
```

```
        drivers.forEach { it.displayInfo(); println() }
```

```
    }
```

```
    fun displayAllCustomers() {
```

```
        println("=== DAFTAR PELANGGAN ===")
```

```
        customers.forEach { it.displayInfo(); println() }
```

```
    }
```

```
    fun displayAllOrders() {
```

```
        println("=== DAFTAR PESANAN ===")
```

```
        orders.forEach { it.displayOrder(); println() }
```

```
    }
```

```
    fun displayRevenueReport() {
```

```
        val totalRev = orders.filter { it.status is OrderStatus.Completed }.sumOf {  
it.totalFare }
```

```
        println("=== LAPORAN PENDAPATAN ===")
```

```
        println("Pendapatan pesanan selesai: ${formatRupiah(totalRev)}")
```

```
    }
```

```
}
```



```
// =====  
// 16. Main.kt  
// =====  
fun main() {  
    val system = TransportSystem("Go-Transport 2024")  
  
    val car = Car("B 1234 XYZ", "Toyota", "Innova", 2021, "Bensin", 4)  
    val motorcycle = Motorcycle("D 5678 ABC", "Honda", "Beat", 2022, 125, true)  
    val truck = Truck("E 9012 DEF", "Hino", "Dutro", 2020, 5.0, 2)  
  
    system.addVehicle(car)  
    system.addVehicle(motorcycle)  
    system.addVehicle(truck)  
  
    val driver1 = Driver("D001", "Andi", "08123456789", car)  
    val driver2 = Driver("D002", "Budi", "08129876543", motorcycle)  
    val driver3 = Driver("D003", "Citra", "08125678901", truck)  
  
    system.addDriver(driver1)  
    system.addDriver(driver2)  
    system.addDriver(driver3)  
  
    val customer1 = Customer("C001", "Dewi", "08134567890", "dewi@email.com", 100000.0)  
    val customer2 = Customer("C002", "Eko", "08135678901", "eko@email.com", 50000.0)  
    val customer3 = Customer("C003", "Fani", "08136789012", "fani@email.com", 200000.0)  
  
    system.addCustomer(customer1)  
    system.addCustomer(customer2)  
    system.addCustomer(customer3)  
  
    println("=== ${system.name} ===\n")  
  
    println("=== TARIF KENDARAAN UNTUK JARAK 20 KM ===")  
  
    println("Mobil: ${formatRupiah(car.calculateFare(20.0))}")
```



```
println("Motor: ${formatRupiah(motorcycle.calculateFare(20.0))}")
```

```
println("Truk: ${formatRupiah(truck.calculateFare(20.0))}\n")
```

```
system.displayAllVehicles()
```

```
system.displayAllDrivers()
```

```
system.displayAllCustomers()
```

```
val order1 = system.createOrder("C001", "D001", "Kampus A", "Mall B", 12.0)
```

```
val order2 = system.createOrder("C002", "D002", "Stasiun", "Kantor", 8.0)
```

```
val order3 = system.createOrder("C003", "D003", "Gudang", "Pelabuhan", 25.0)
```

```
system.displayAllOrders()
```

```
println("=== PEMBAYARAN PESANAN 1 ===")
```

```
val pay1 = system.processPayment("ORD-001", QRIS("12345678901234"),  
order1!!.totalFare)
```

```
println(pay1.display())
```

```
println("\n=== PEMBAYARAN PESANAN 2 ===")
```

```
val pay2 = system.processPayment("ORD-002", Cash(), 10000.0)
```

```
println(pay2.display())
```

```
customer2.topUp(20000.0)
```

```
println("Saldo Eko setelah top up: ${formatRupiah(customer2.balance)}")
```

```
val pay3 = system.processPayment("ORD-002", CreditCard("1234567890123456"),  
order2!!.totalFare)
```

```
println(pay3.display())
```

```
val res1 = system.completeOrder("ORD-001")
```

```
val res3 = system.cancelOrder("ORD-003", "Hujan deras")
```

```
println("\nOrder 1 selesai: $res1")
```

```
println("Order 3 dibatalkan: $res3\n")
```

```
system.displayAllOrders()
```

```
system.displayRevenueReport()
```

```
println("\n=== DEMONSTRASI SOAL 4 ===")
```



```
println("Tarif Mobil untuk 10 km: ${formatRupiah(car.calculateFare(10.0))}")
```

```
println("Tarif Motor untuk 10 km: ${formatRupiah(motorcycle.calculateFare(10.0))}")
```

```
println("Tarif Truk untuk 10 km: ${formatRupiah(truck.calculateFare(10.0))}")
```

```
val methods: List<PaymentMethod> = listOf(
```

```
    CreditCard("1234567890123456"),
```

```
    QRIS("12345678901234"),
```

```
    Cash()
```

```
)
```

```
methods.forEach { m ->
```

```
    val res = m.processPayment(50000.0)
```

```
    println("${m.name}: ${res.display()}")
```

```
}
```

```
println("\n=== DEMONSTRASI POLIMORFISME ===")
```

```
val vList: List<Vehicle> = listOf(car, motorcycle, truck)
```

```
vList.forEach { v ->
```

```
    println("${v.getType()}                                ${v.plateNumber}:  
${formatRupiah(v.calculateFare(15.0))}")
```

```
    when (v) {
```

```
        is Car -> println("Bahan bakar : ${v.fuelType}")
```

```
        is Motorcycle -> println("Kapasitas mesin : ${v.engineCapacity} cc")
```

```
        is Truck -> println("Kapasitas muatan : ${v.loadCapacity} ton")
```

```
    }
```

```
}
```

```
val safeCast = driver1.vehicle as? Car
```

```
println("\nHasil casting aman kendaraan D001: ${safeCast?.fuelType ?: "Bukan  
Mobil"}")
```

```
println("\n=== DEMONSTRASI SEALED CLASS ===")
```

```
val statuses: List<OrderStatus> = listOf(
```

```
    OrderStatus.Waiting,
```

```
    OrderStatus.OnGoing,
```

```
    OrderStatus.Completed,
```

```
    OrderStatus.Cancelled("Perubahan rencana")
```

```
)
```



```
statuses.forEach { st ->
    when (st) {
        is OrderStatus.Waiting -> println(st.display())
        is OrderStatus.OnGoing -> println(st.display())
        is OrderStatus.Completed -> println(st.display())
        is OrderStatus.Cancelled -> println(st.display())
    }
}
```